

Software Engineering in Practice - Assignment 1

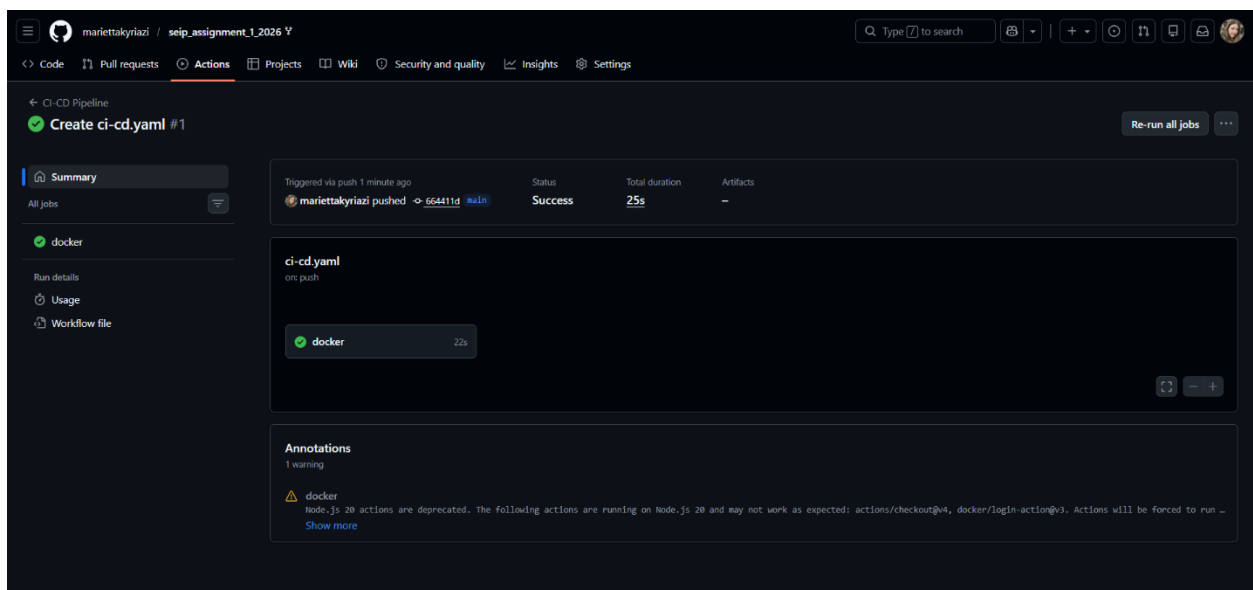
Name: Marietta Kyriazi

GitHub: mariettakyriazi

1. GitHub Repository Link

https://github.com/mariettakyriazi/seip_assignment_1_2026

2. CI/CD Proof



3. Cluster State Proof

```
PS C:\Users\marie\seip_assignment_1_2026> kubectl get all -n default
```

NAME	READY	STATUS	RESTARTS	AGE
pod/echo-api-deployment-cb7d47c58-5xrj2	1/1	Running	0	3m8s
pod/echo-api-deployment-cb7d47c58-r2j2d	1/1	Running	0	3m8s
pod/echo-api-deployment-cb7d47c58-tpvds	1/1	Running	0	3m8s

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/echo-api-service	ClusterIP	10.100.1.77	<none>	80/TCP	3m8s
service/kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	4m22s

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/echo-api-deployment	3/3	3	3	3m8s

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/echo-api-deployment-cb7d47c58	3	3	3	3m8s

```
PS C:\Users\marie\seip_assignment_1_2026> |
```

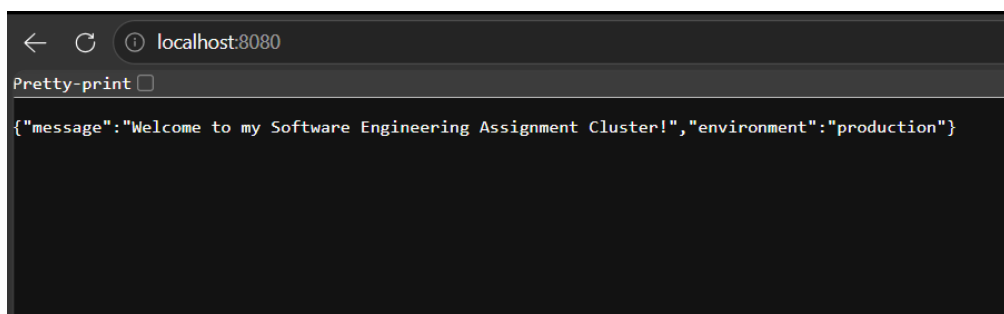
```
PS C:\Users\marie\seip_assignment_1_2026> kubectl get configmap,secret
```

NAME	DATA	AGE
configmap/echo-api-config	2	70s
configmap/kube-root-ca.crt	1	2m19s

NAME	TYPE	DATA	AGE
secret/echo-api-secret	Opaque	1	70s

```
PS C:\Users\marie\seip_assignment_1_2026>
```

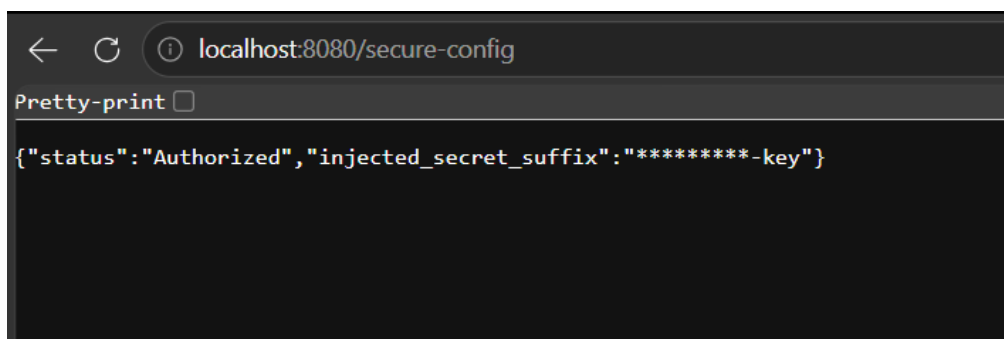
4. Application Verification Proof



← ↻ ⓘ localhost:8080

Pretty-print ☐

```
{\"message\": \"Welcome to my Software Engineering Assignment Cluster!\", \"environment\": \"production\"}
```



← ↻ ⓘ localhost:8080/secure-config

Pretty-print ☐

```
{\"status\": \"Authorized\", \"injected_secret_suffix\": \"*****-key\"}
```

5. AI Reflection & Future Outlook

AI Reflection

I used ChatGPT to better understand Docker, GitHub Actions and Kubernetes concepts. It helped me understand the YAML files and troubleshoot some configuration issues.

However, I still had to test everything locally and solve problems related to Docker Desktop, WSL and Minikube.

Future Outlook

If I had more time, I would further improve the project by adding:

- an Ingress Controller
- monitoring with Prometheus
- automated security scanning
- a GitOps workflow with ArgoCD